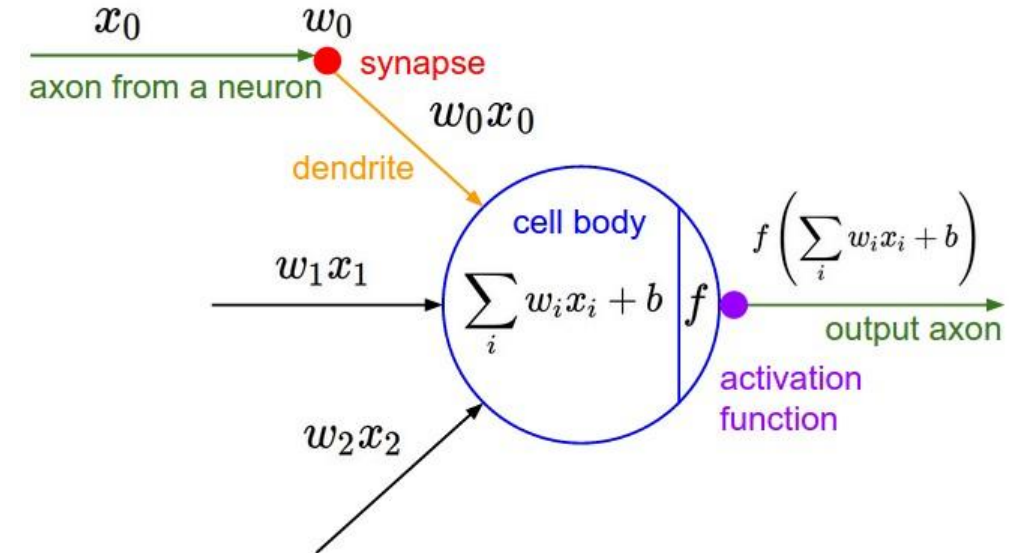
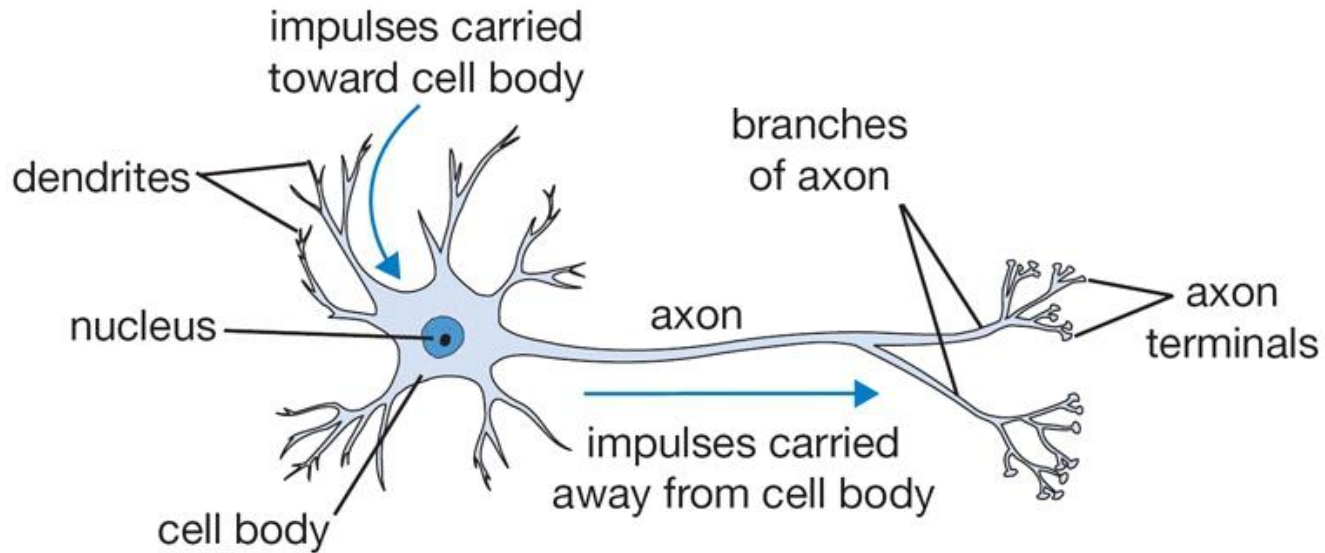
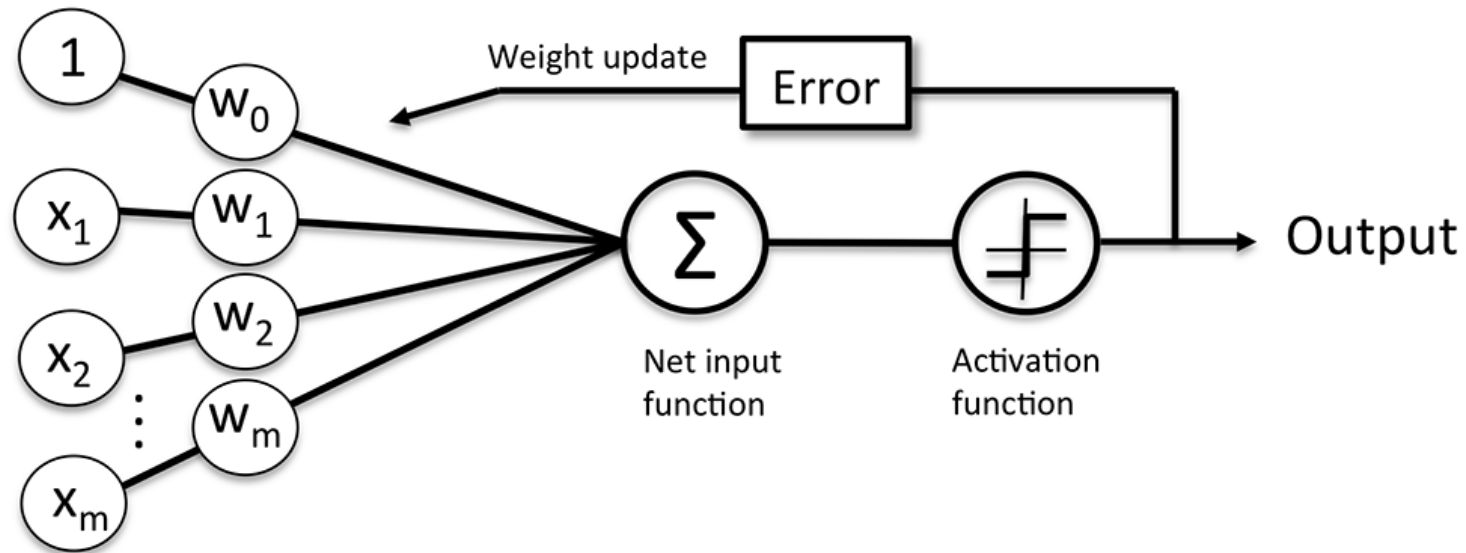


Neural Networks

Biological Idea

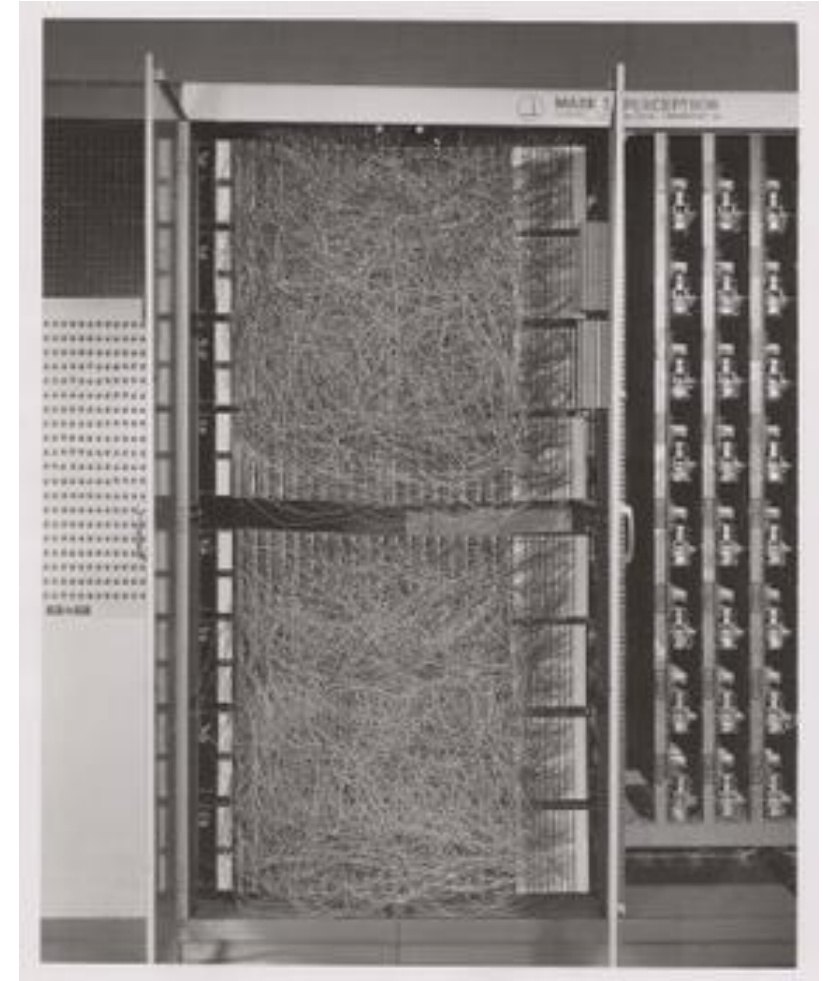


Perceptron (Frank Rosenblatt, 1957)

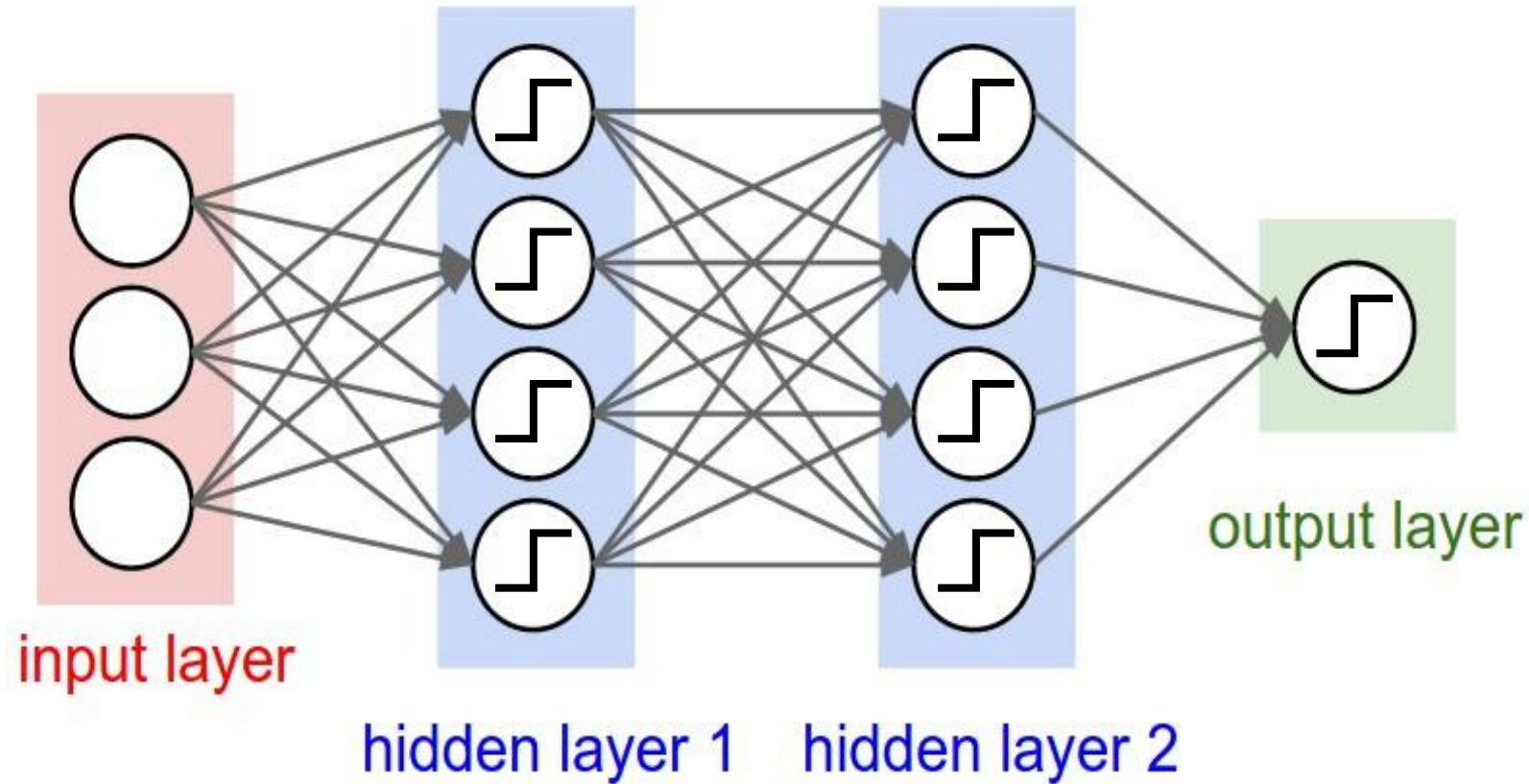


$$\text{sign}(\mathbf{w}^T \mathbf{x}_n) \neq y_n$$

$$\mathbf{w} \leftarrow \mathbf{w} + y_n \mathbf{x}_n$$



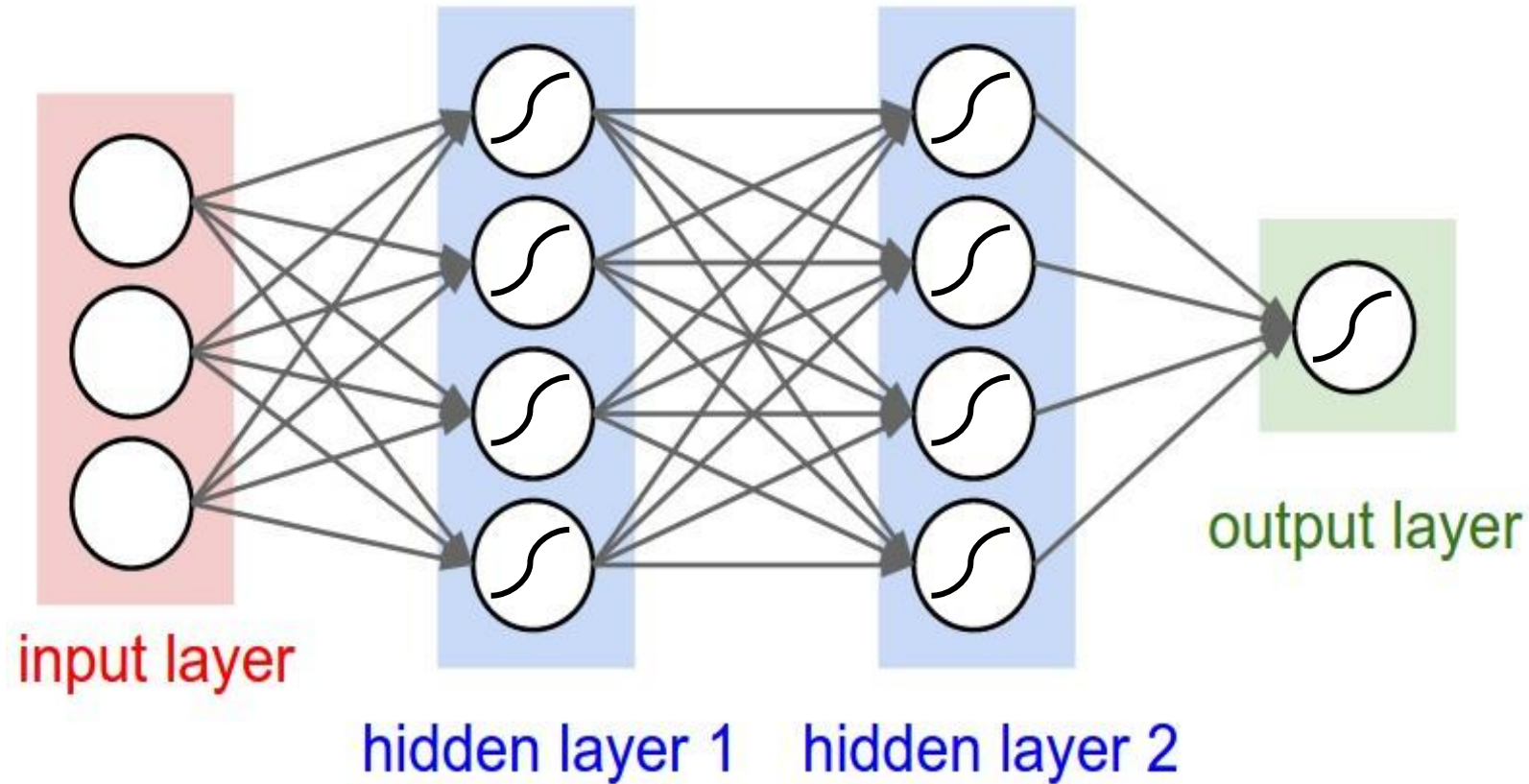
Multi-Layer Perceptron (MLP)



Perceptron is very limited

Marvin Minsky and Seymour Papert, Perceptrons: An Introduction to Computational Geometry

Multi-Layer Perceptron (MLP)



Perceptron is very limited

Marvin Minsky and Seymour Papert, Perceptrons: An Introduction to Computational Geometry

Universal Approximation Theorem

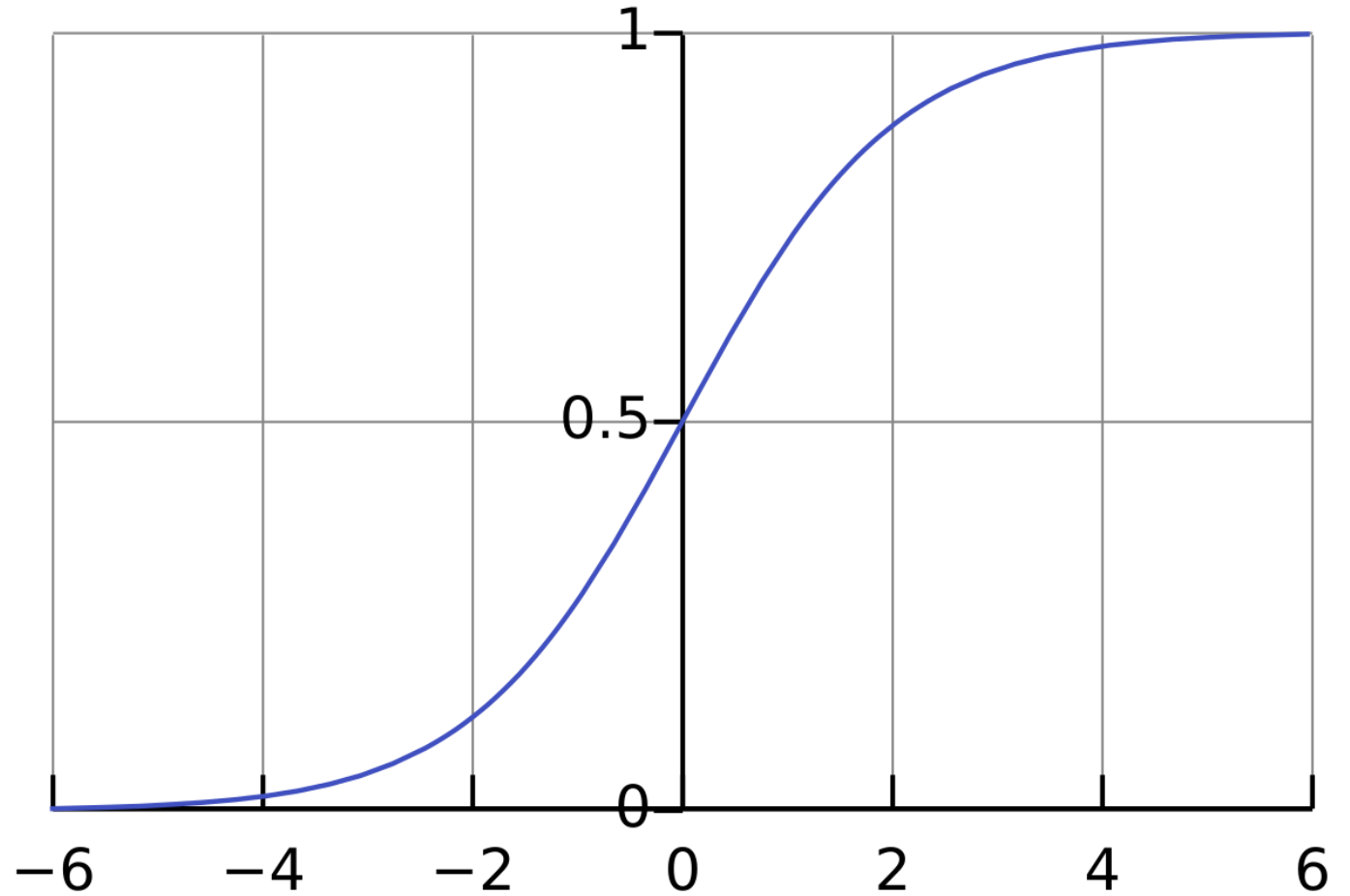
Suppose $\sigma: \mathbb{R} \rightarrow \mathbb{R}$ is nonconstant, bounded, nondecreasing and continuous, and let $S \subseteq \mathbb{R}^d$ be closed and bounded. Then for any continuous function $f: S \rightarrow \mathbb{R}$ and any $\epsilon > 0$, there exist a neural network (perceptron) with one hidden layer containing finitely many nodes, which we can write

$$h(\mathbf{x}) = \sum_{j=1}^k c_j \sigma(\mathbf{w}_j^T \mathbf{x})$$

such that $|h(\mathbf{x}) - f(\mathbf{x})| < \epsilon$

Logistic Function (Sigmoid)

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$
$$\sigma(-x) = 1 - \sigma(x)$$



Logistic Regression

We can convert the “raw” linear score into probabilities:

$$g(\mathbf{x}) = \sigma(\mathbf{w}^T \mathbf{x})$$

We use the sigmoid function to output a probability distribution $P(\hat{Y})$ over the classes:

$$P(\hat{Y} = y \mid \mathbf{x}, \mathbf{w}) = \begin{cases} \sigma(\mathbf{w}^T \mathbf{x}), & y = 1 \\ 1 - \sigma(\mathbf{w}^T \mathbf{x}), & y = -1 \end{cases} \quad \text{or} \quad P(\hat{Y} = y \mid \mathbf{x}, \mathbf{w}) = \sigma(y\mathbf{w}^T \mathbf{x})$$

$$\hat{y} = \max_k P(\hat{Y} = k \mid \mathbf{x}, \mathbf{w}) = \begin{cases} 1, & \sigma(\mathbf{w}^T \mathbf{x}) \geq 0.5 \\ 0, & \text{otherwise} \end{cases}$$

Equivalently, we classify $\mathbf{x}$ as $\hat{y} = \begin{cases} 1, & \mathbf{w}^T \mathbf{x} \geq 0.5 \\ 0, & \text{otherwise} \end{cases}$

Negative Log Likelihood Loss

Likelihood: $P(\hat{Y}_1 = y_1, \dots, \hat{Y}_n = y_n \mid \mathbf{x}_1, \dots, \mathbf{x}_n, \mathbf{w})$

$$\prod_i^N P(y_i \mid \mathbf{x}_i, \mathbf{w}) = \prod_i^N \sigma(y_i \mathbf{w}^T \mathbf{x}_i)$$

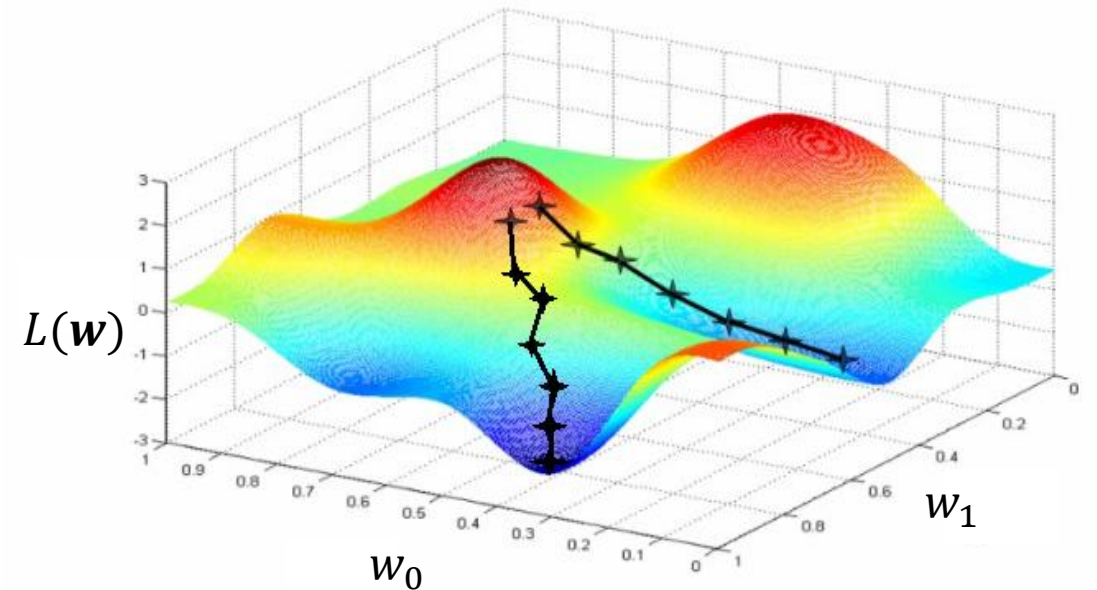
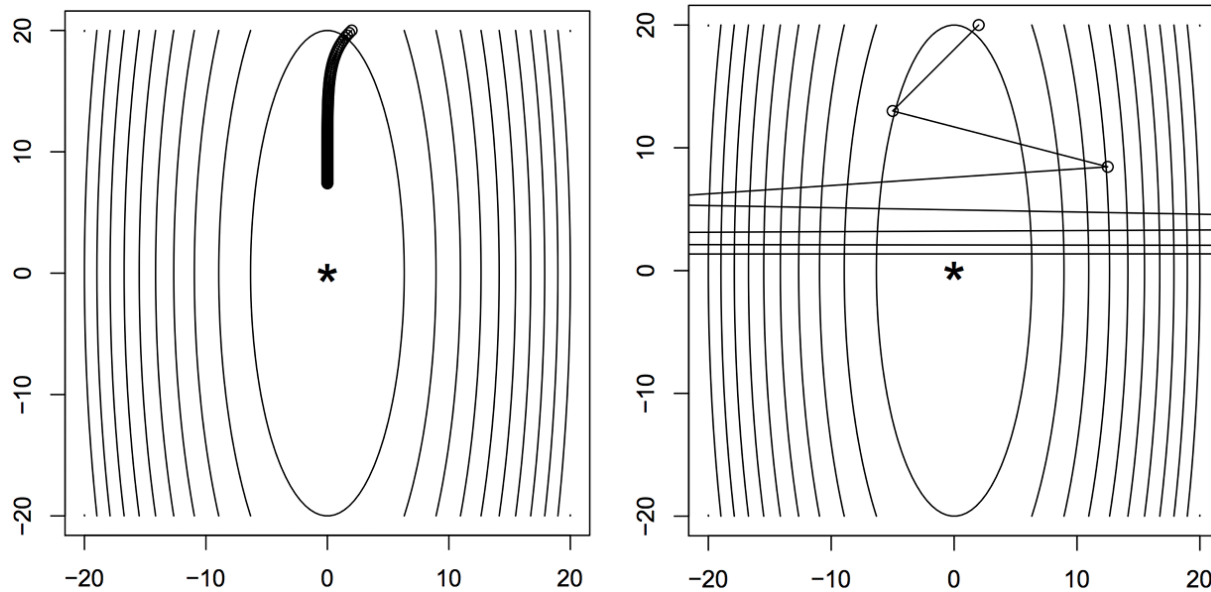
$$L(\mathbf{w}) = -\frac{1}{N} \ln \left(\prod_{i=1}^N \sigma(y_i \mathbf{w}^T \mathbf{x}_i) \right) = \frac{1}{N} \sum_{i=1}^N \ln \left(\frac{1}{\sigma(y_i \mathbf{w}^T \mathbf{x}_i)} \right)$$

$$= \frac{1}{N} \sum_{i=1}^N \ln \left(1 + e^{-y_i \mathbf{w}^T \mathbf{x}_i} \right)$$

(Standard or Batch) Gradient Descent

$$\mathbf{w}_{t+1} \leftarrow \mathbf{w}_t - \alpha_t \nabla f(\mathbf{w}_t), \quad \nabla f(\mathbf{w}) = \frac{1}{|D|} \sum_{i=1}^{|D|} \nabla f_i(\mathbf{w})$$

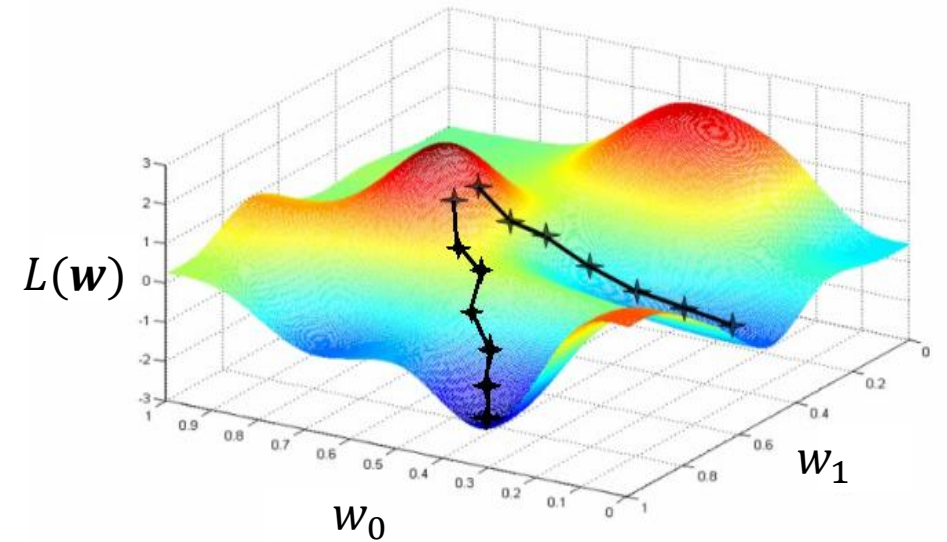
- Deterministic
- Can easily get stuck at local minima and saddle points
- Computationally expensive



Stochastic Gradient Descent

$$\mathbf{w}_{t+1} \leftarrow \mathbf{w}_t - \alpha_t \nabla f_i(\mathbf{w}_t)$$

- Unbiased estimate of the true gradient: $E_i[\nabla f_i(\mathbf{w})] = \nabla f(\mathbf{w})$
- Has high variance
- Often requires a significantly higher number of updates



An **epoch** is a measure of time it is defined as the number of iterations in order to traverse the training data once

Mini-Batch Gradient Descent

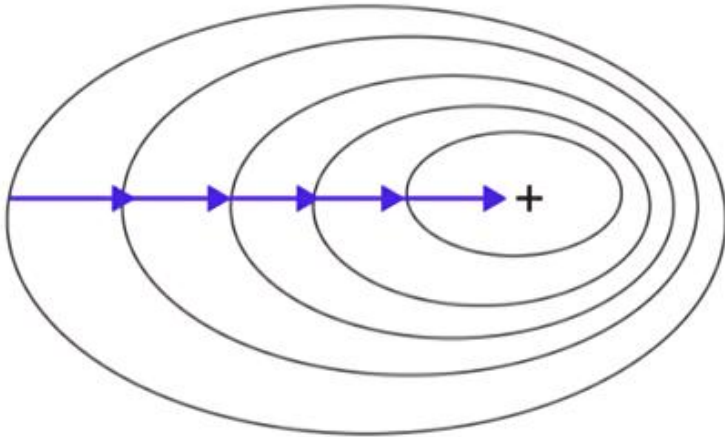
$$\mathbf{w}_{t+1} \leftarrow \mathbf{w}_t - \alpha_t \nabla f_B(\mathbf{w}_t), \quad \nabla f_B(\mathbf{w}) = \frac{1}{|B|} \sum_B \nabla f_k(\mathbf{w})$$

- Significantly more computationally efficient
- Capturing the stability of batch gradient
- In practice, mini-batch gradient descent is more effective than SGD and batch gradient descent

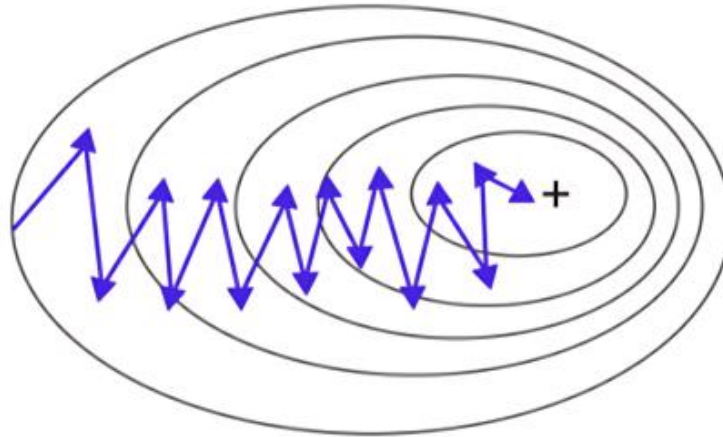
An **epoch** is a measure of time it is defined as the number of iterations in order to traverse the training data once

Batch vs Stochastic vs Mini-Batch

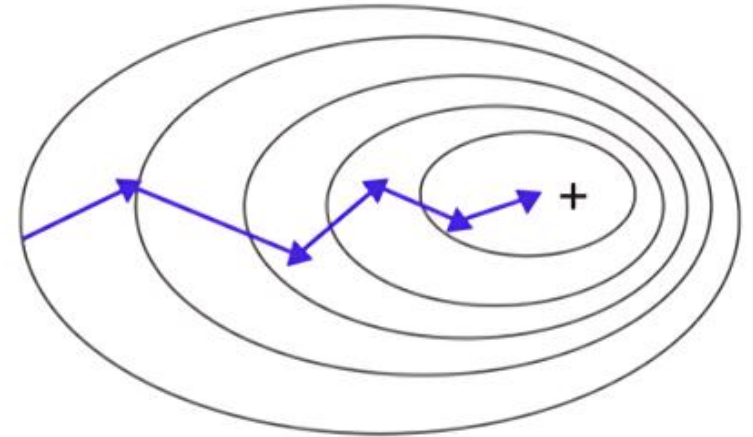
Batch Gradient Descent



Stochastic Gradient Descent



Mini-Batch Gradient Descent



Formula for Logistic Regression

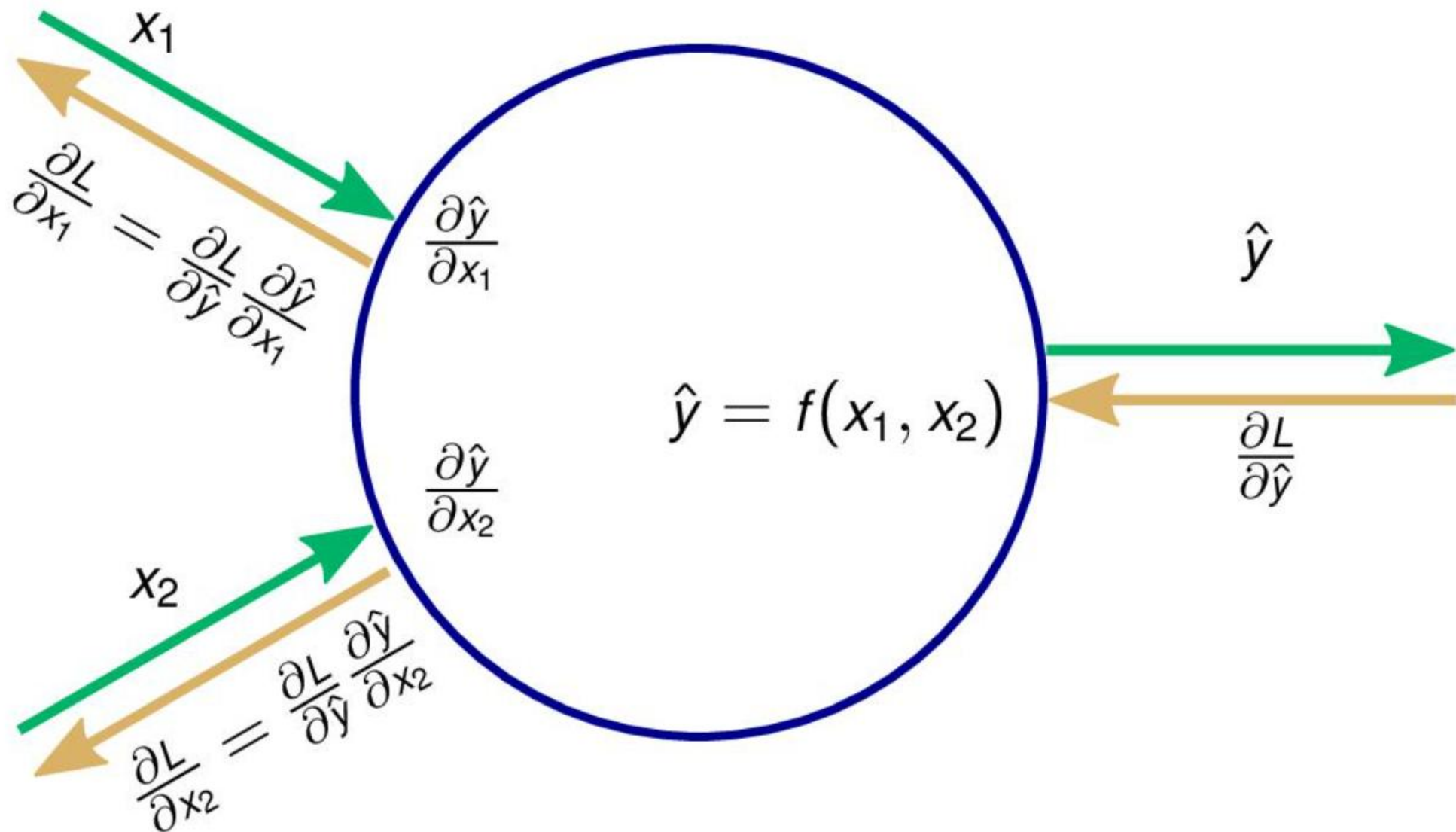
Batch:

$$w_{t+1} \leftarrow w_t - \alpha \left(-\frac{1}{|D|} \sum_{i=1}^{|D|} \frac{y_i x_i}{1 + e^{y_i w^T x_i}} \right)$$

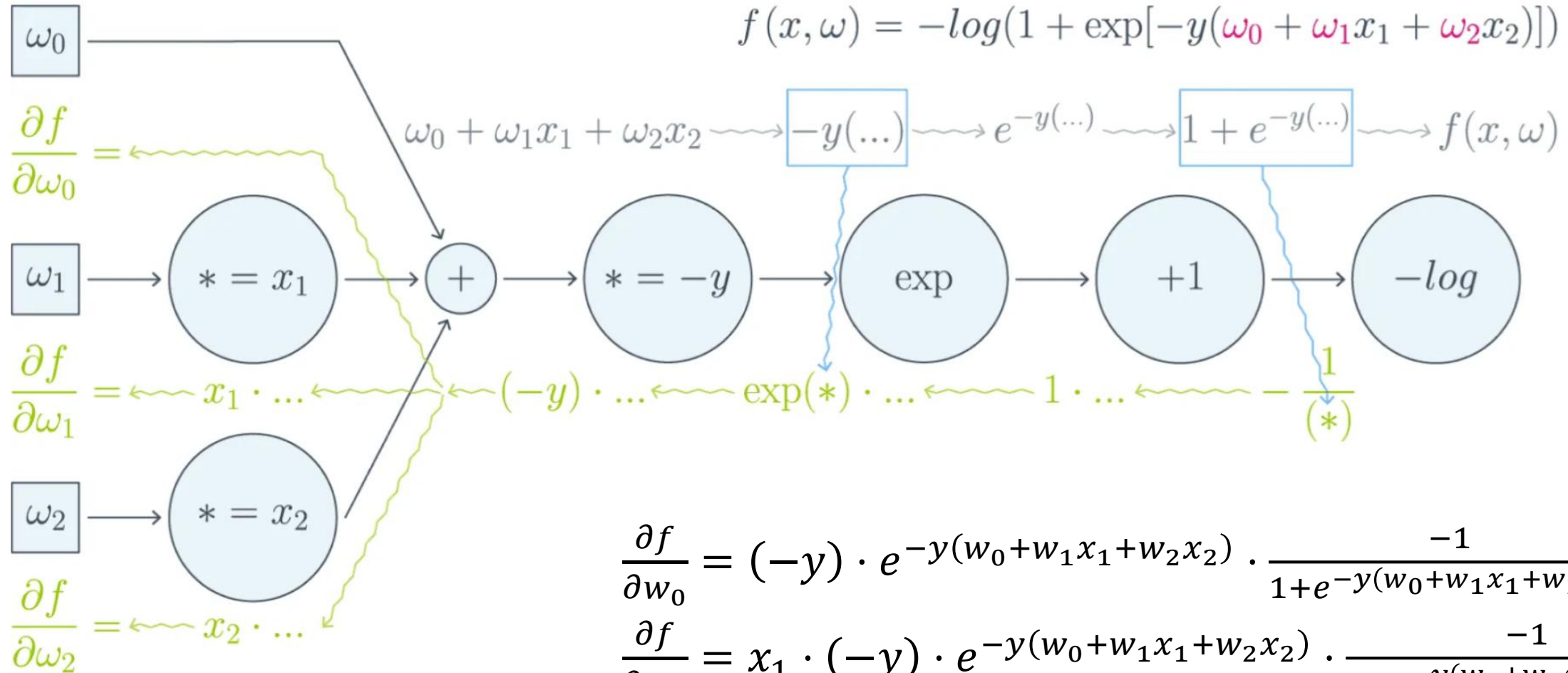
Mini-Batch:

$$w_{t+1} \leftarrow w_t - \alpha \left(-\frac{1}{|B|} \sum_{x_i \in B} \frac{y_i x_i}{1 + e^{y_i w^T x_i}} \right)$$

Backpropagation: Idea



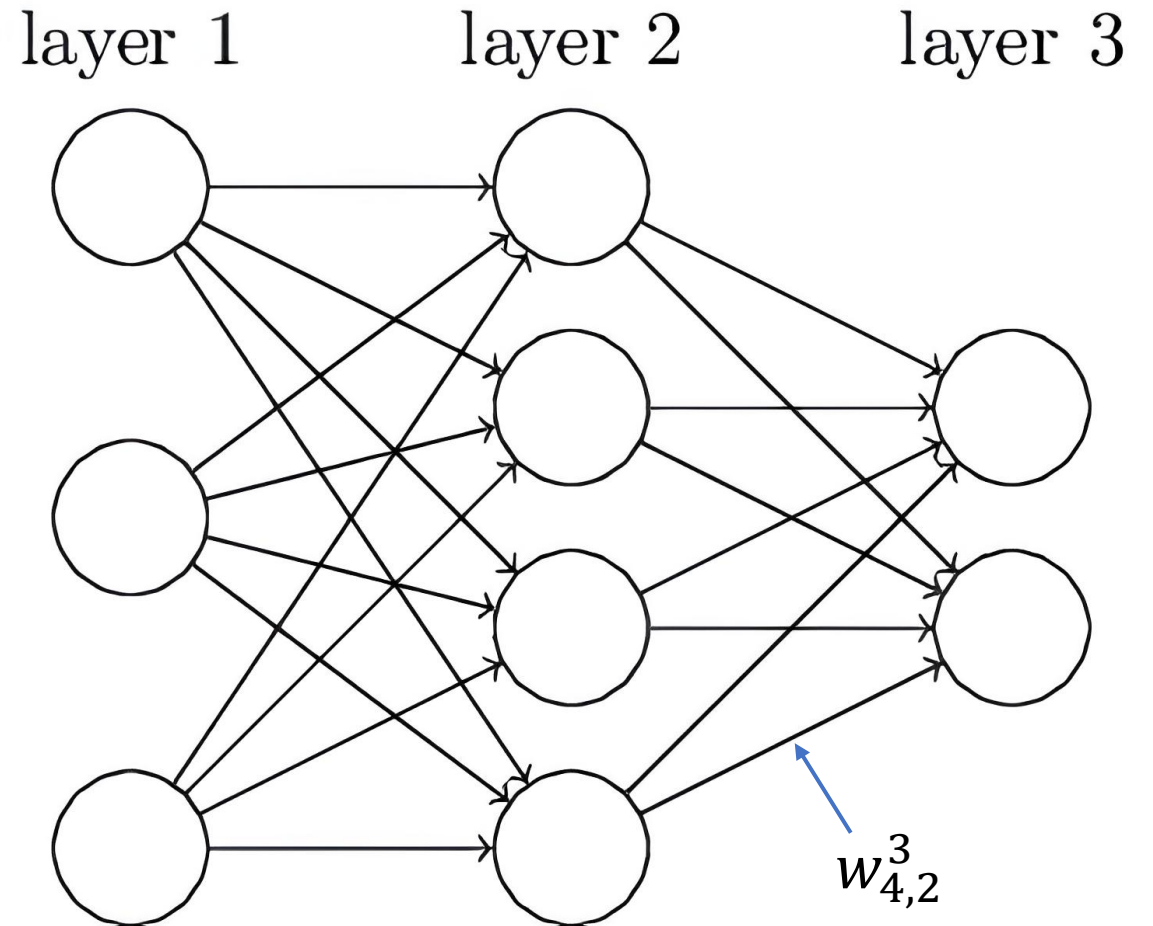
Backpropagation: Logistic Regression



$$\begin{aligned} \frac{\partial f}{\partial \omega_0} &= (-y) \cdot e^{-y(\omega_0 + \omega_1 x_1 + \omega_2 x_2)} \cdot \frac{-1}{1 + e^{-y(\omega_0 + \omega_1 x_1 + \omega_2 x_2)}} \\ \frac{\partial f}{\partial \omega_1} &= x_1 \cdot (-y) \cdot e^{-y(\omega_0 + \omega_1 x_1 + \omega_2 x_2)} \cdot \frac{-1}{1 + e^{-y(\omega_0 + \omega_1 x_1 + \omega_2 x_2)}} \\ \frac{\partial f}{\partial \omega_2} &= x_2 \cdot (-y) \cdot e^{-y(\omega_0 + \omega_1 x_1 + \omega_2 x_2)} \cdot \frac{-1}{1 + e^{-y(\omega_0 + \omega_1 x_1 + \omega_2 x_2)}} \end{aligned}$$

Backpropagation

$w_{j,k}^l$ – is the weight between neuron j in layer $l - 1$ and neuron k in layer l



Backpropagation

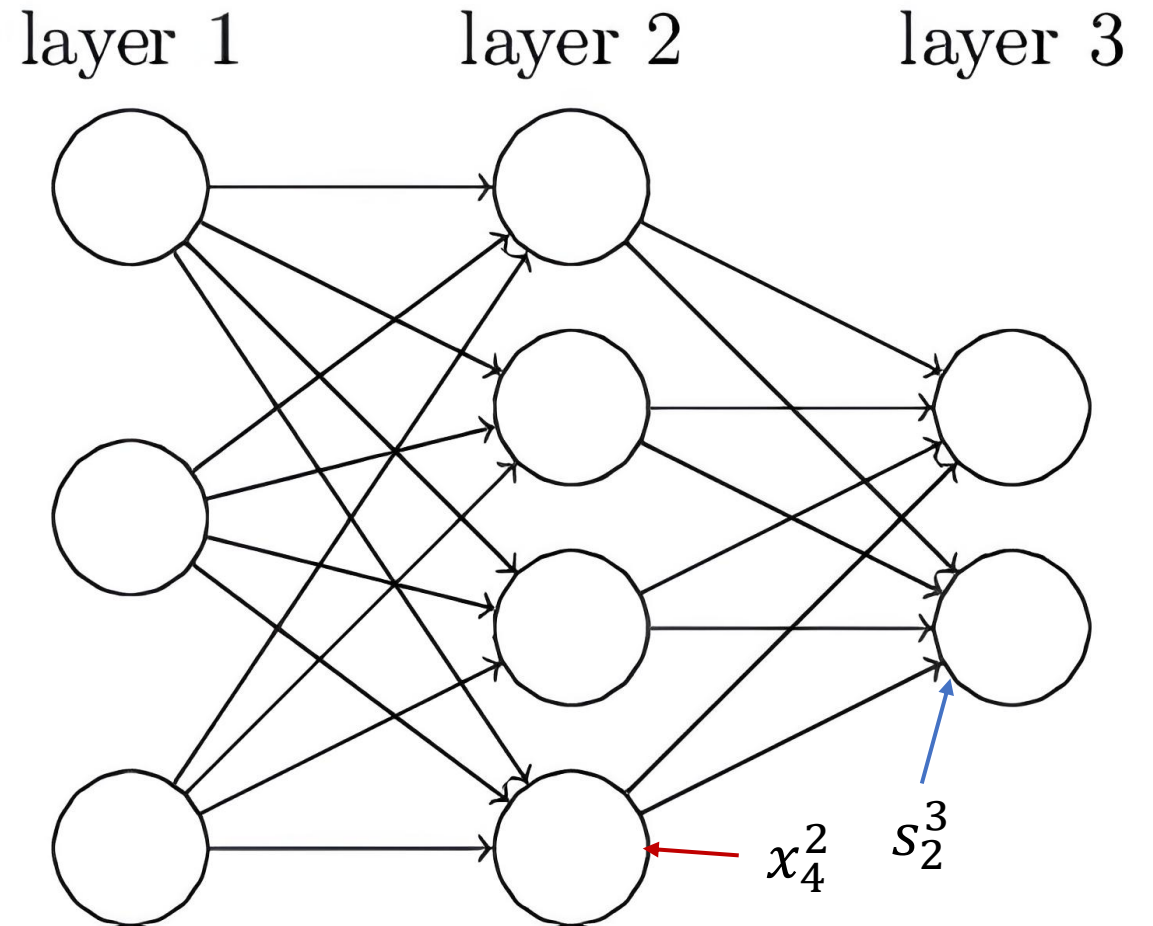
s_j^l – incoming “signal” to neuron j in layer l

$$s_j^l = \sum w_{i,j}^l x_i^{l-1}$$

x_j^l – outgoing “signal” after the activation function

$$x_j^l = \sigma(s_j^l) = \sigma(\sum w_{i,j}^l x_i^{l-1})$$

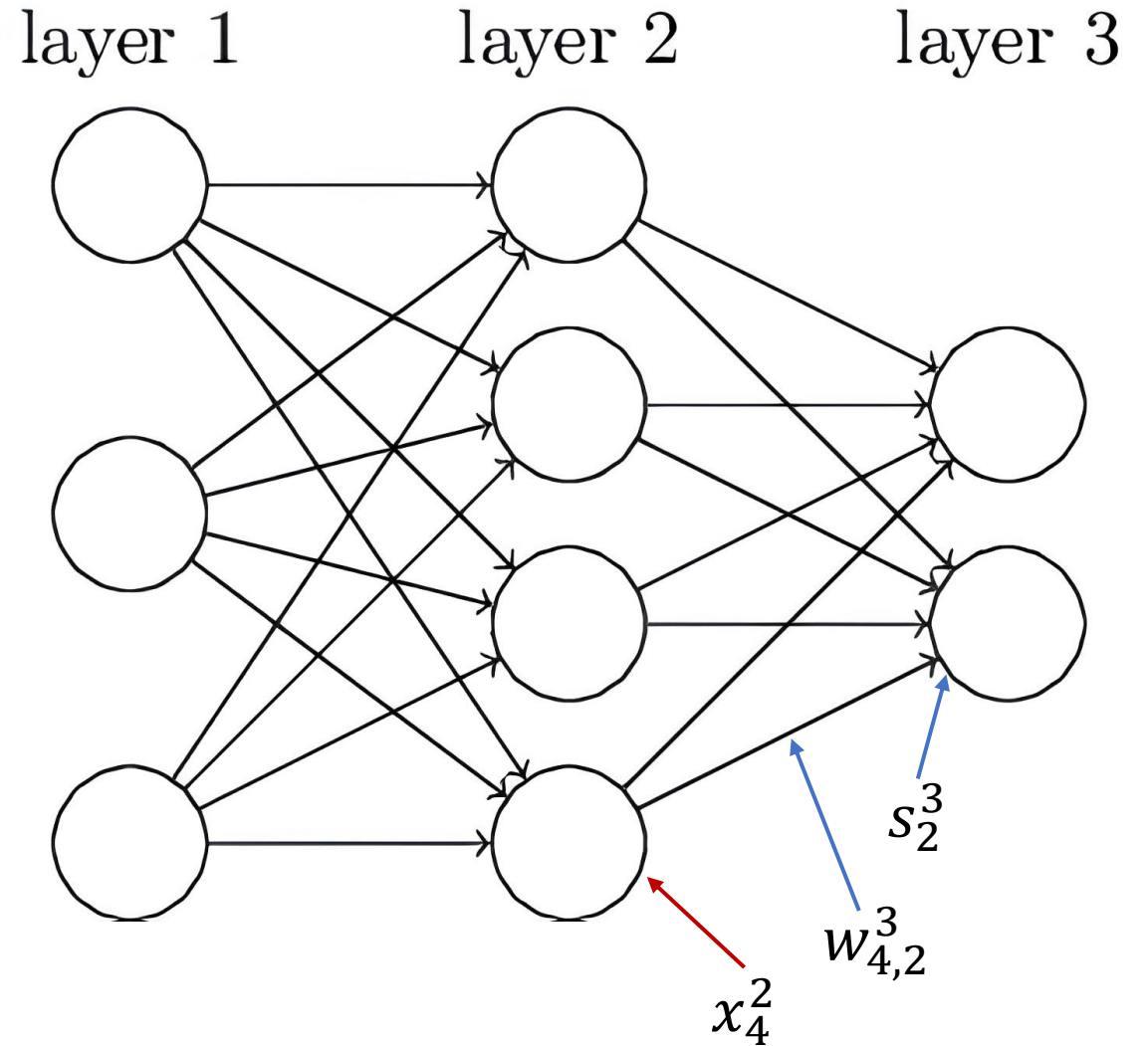
$$x_j^1 = x_j$$



Backpropagation

$$\nabla_{w_{i,j}^l} L(w): \frac{\partial L(w)}{\partial w_{i,j}^l} = \frac{\partial L(w)}{\partial s_j^l} \times \frac{\partial s_j^l}{\partial w_{i,j}^l}$$

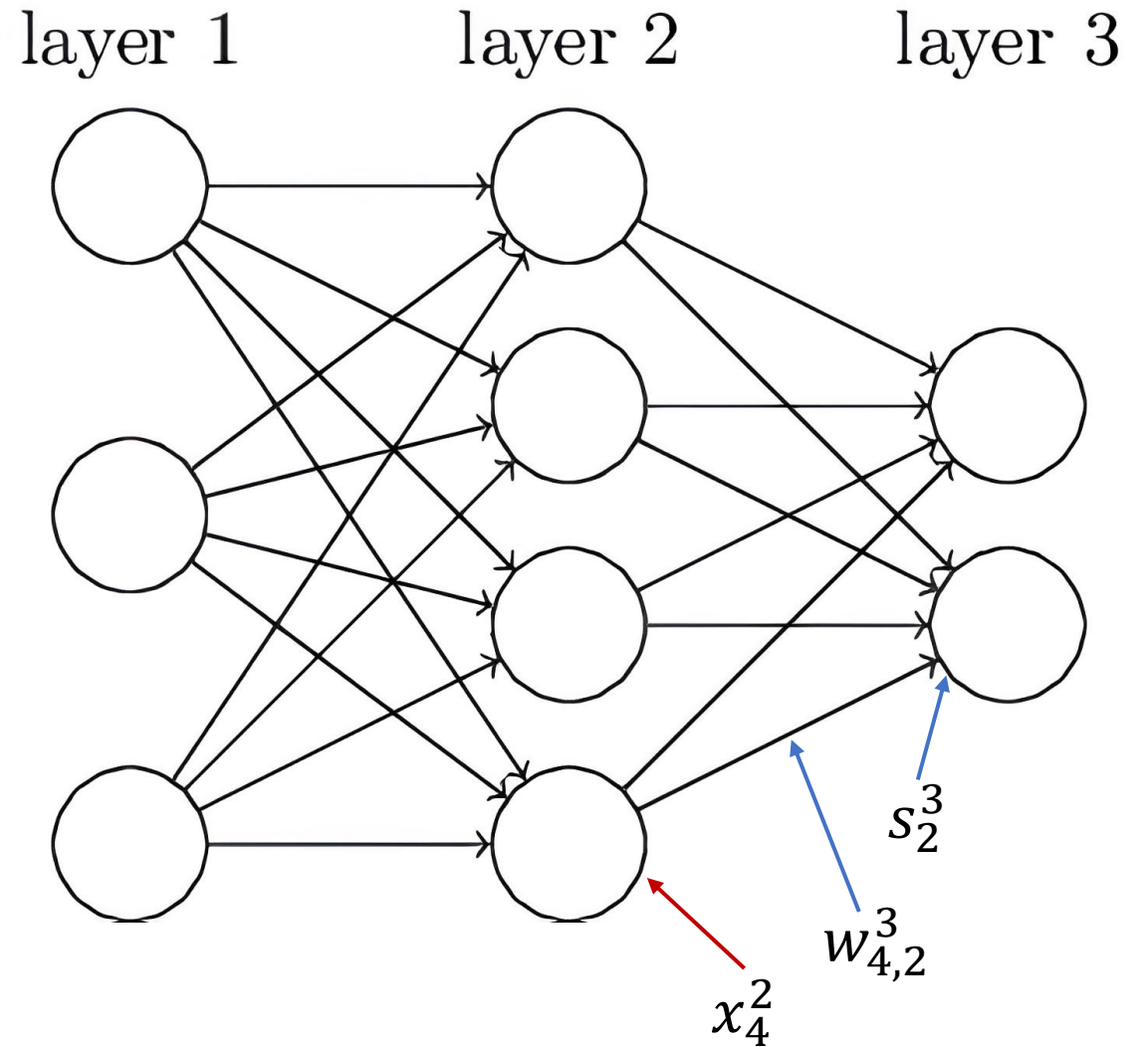
$$\frac{\partial s_j^l}{\partial w_{i,j}^l} = x_i^{l-1}, \quad \frac{\partial L(w)}{\partial s_j^l} = \delta_j^l$$



Backpropagation

Last layer:

$$\delta_i^{Last} = \frac{\partial L(w)}{\partial s_i^{Last}}, \quad L(w) = f(x^{Last})$$
$$x_i^{Last} = \sigma(s_i^{Last})$$



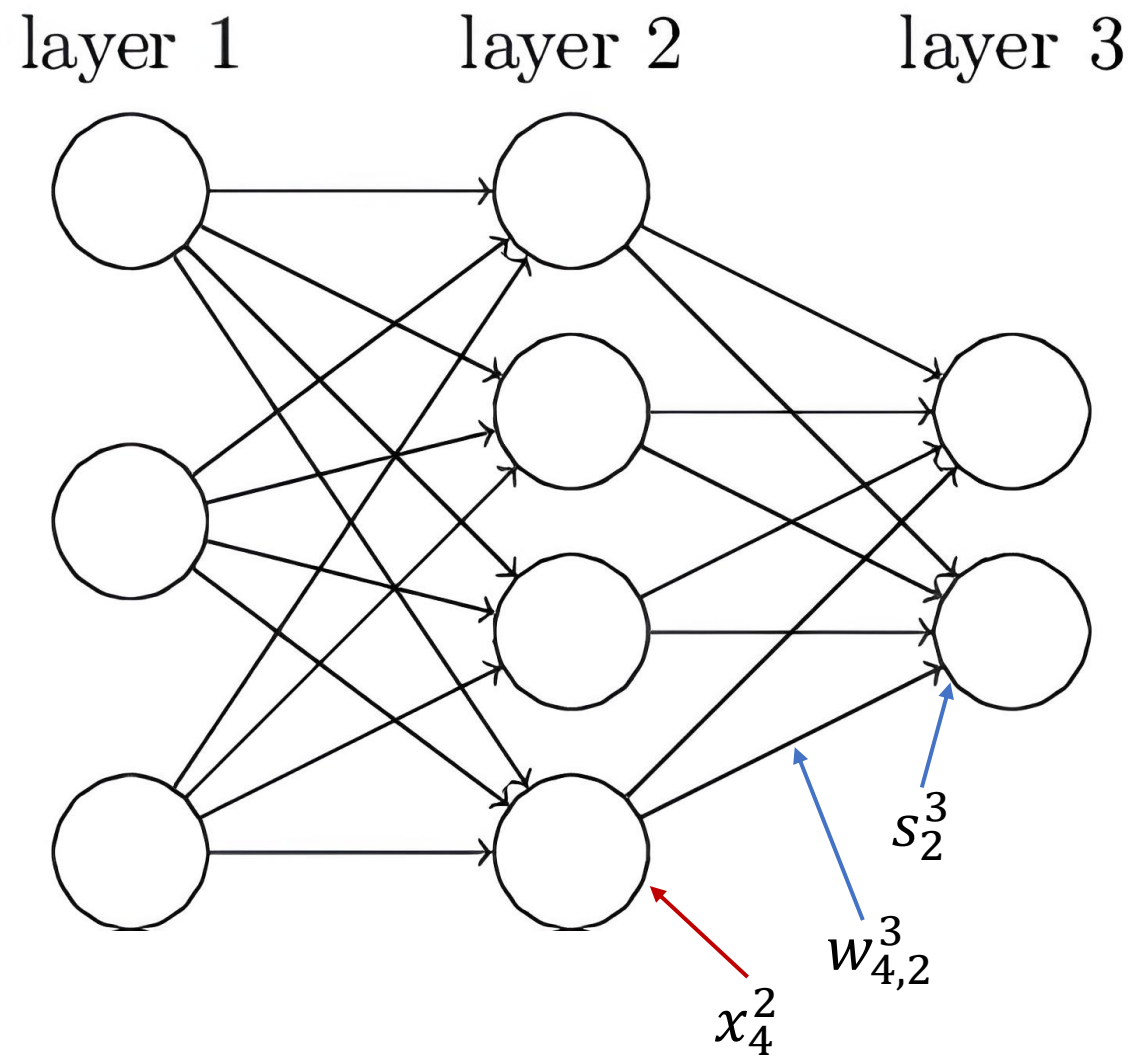
Backpropagation

Last layer:

$$\delta_i^{Last} = \frac{\partial L(w)}{\partial s_i^{Last}}, \quad L(w) = f(x^{Last})$$
$$x_i^{Last} = \sigma(s_i^{Last})$$

Previous layer:

$$\delta_i^{l-1} = \frac{\partial L(w)}{\partial s_i^{l-1}} = \sum_j \frac{\partial L(w)}{\partial s_i^l} \times \frac{\partial s_i^l}{\partial x_i^{l-1}} \times \frac{\partial x_i^{l-1}}{\partial s_i^{l-1}}$$
$$= \sum_j \delta_j^l \times w_{i,j}^l \times \sigma'(s_i^{l-1})$$

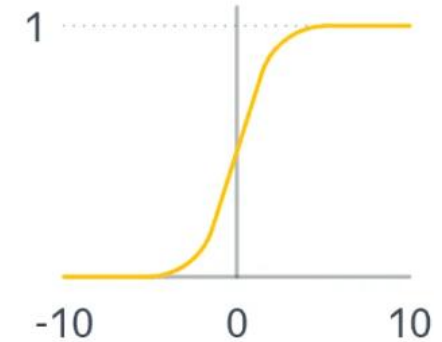


Backpropagation

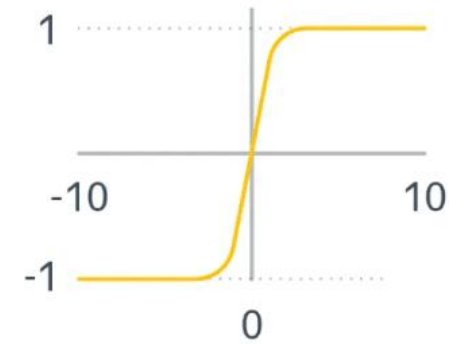
1. Initialize weights randomly*
2. Forward pass: calculate x and s
3. Backward pass: calculate δ
4. $w_{i,j}^l \leftarrow w_{i,j}^l - \alpha x_i^{l-1} \delta_j^l$
5. Repeat from 2

Activation Functions

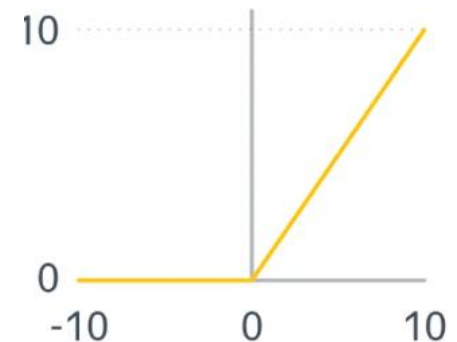
Sigmoid: $\sigma(s) = \frac{1}{1+e^{-s}}$



Tanh: $\sigma(s) = \frac{e^s - e^{-s}}{e^s + e^{-s}}$



Rectified Linear Unit (ReLU): $\sigma(s) = \max(0, s)$

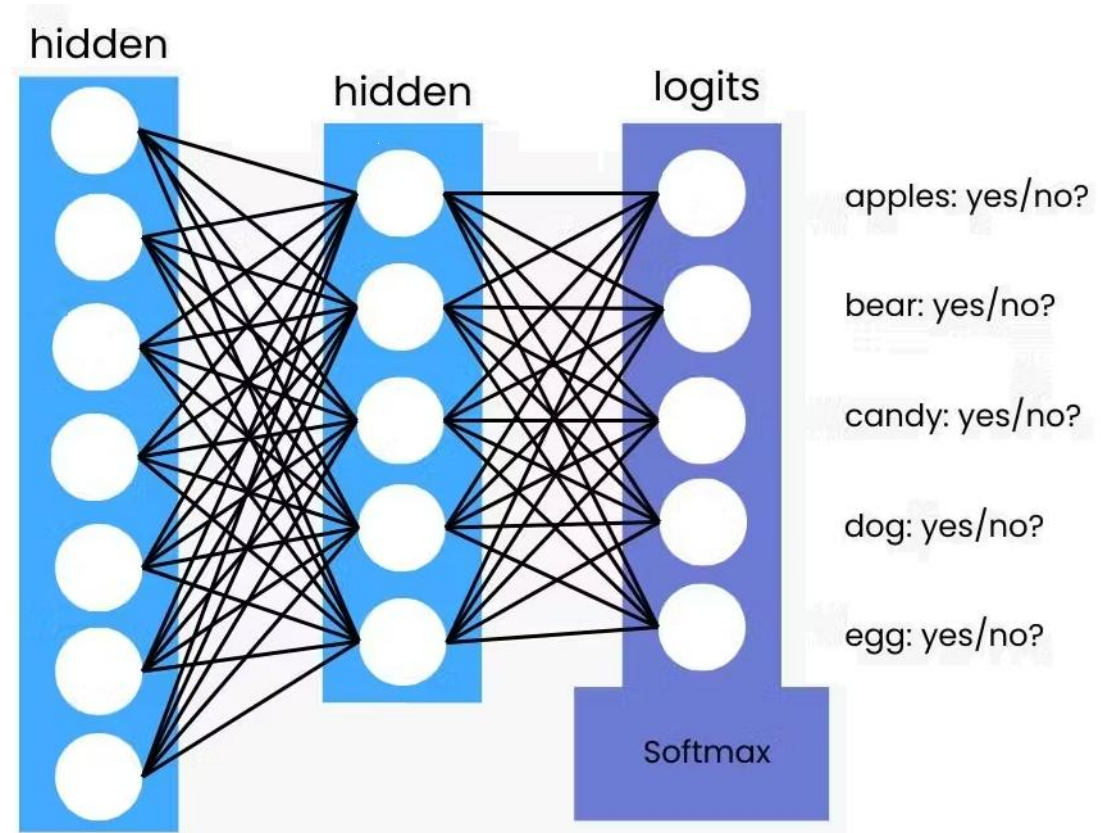


SoftMax and Cross-Entropy

$$\text{softmax}(s_1, \dots, s_n) = \left(\frac{e^{s_1}}{\sum_k e^{s_k}}, \dots, \frac{e^{s_n}}{\sum_k e^{s_k}} \right)$$

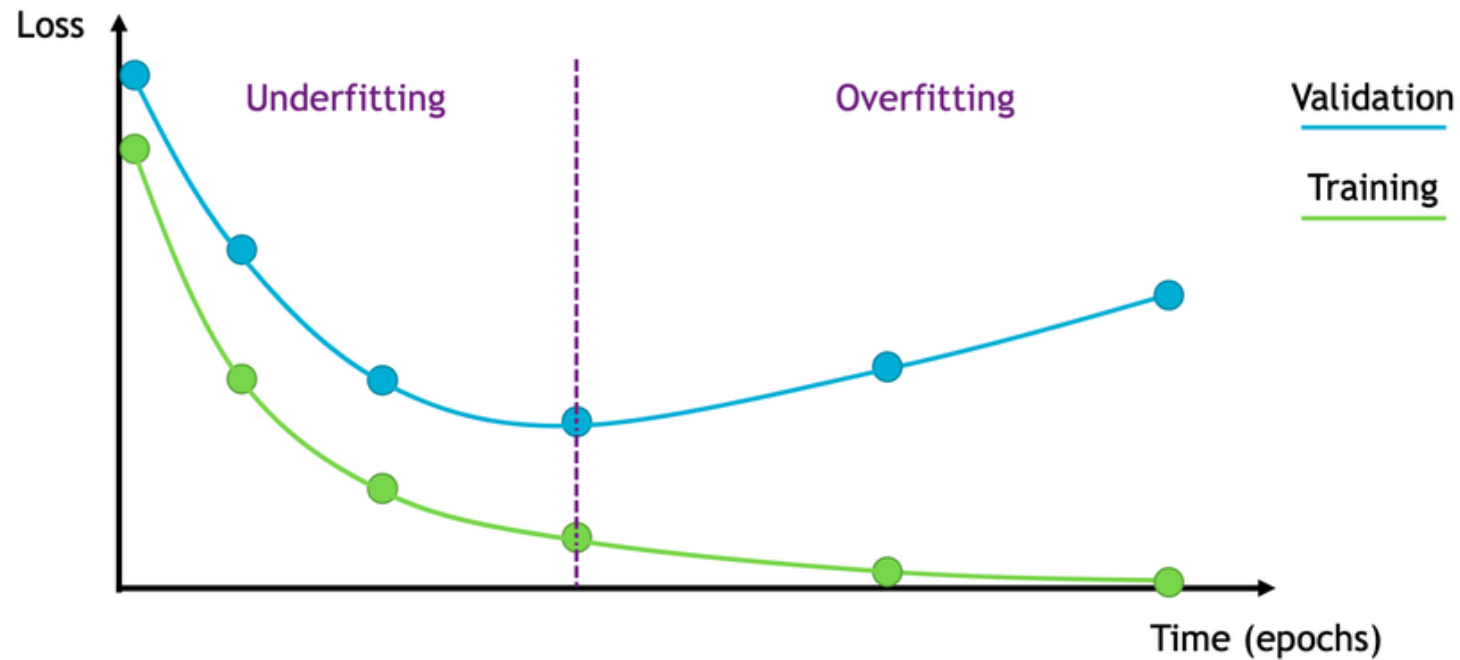
$$x^{Last} = \text{softmax}(s^{Last})$$

$$L(w) = - \sum \mathbb{I}[y = i] \log x_i^{Last}$$

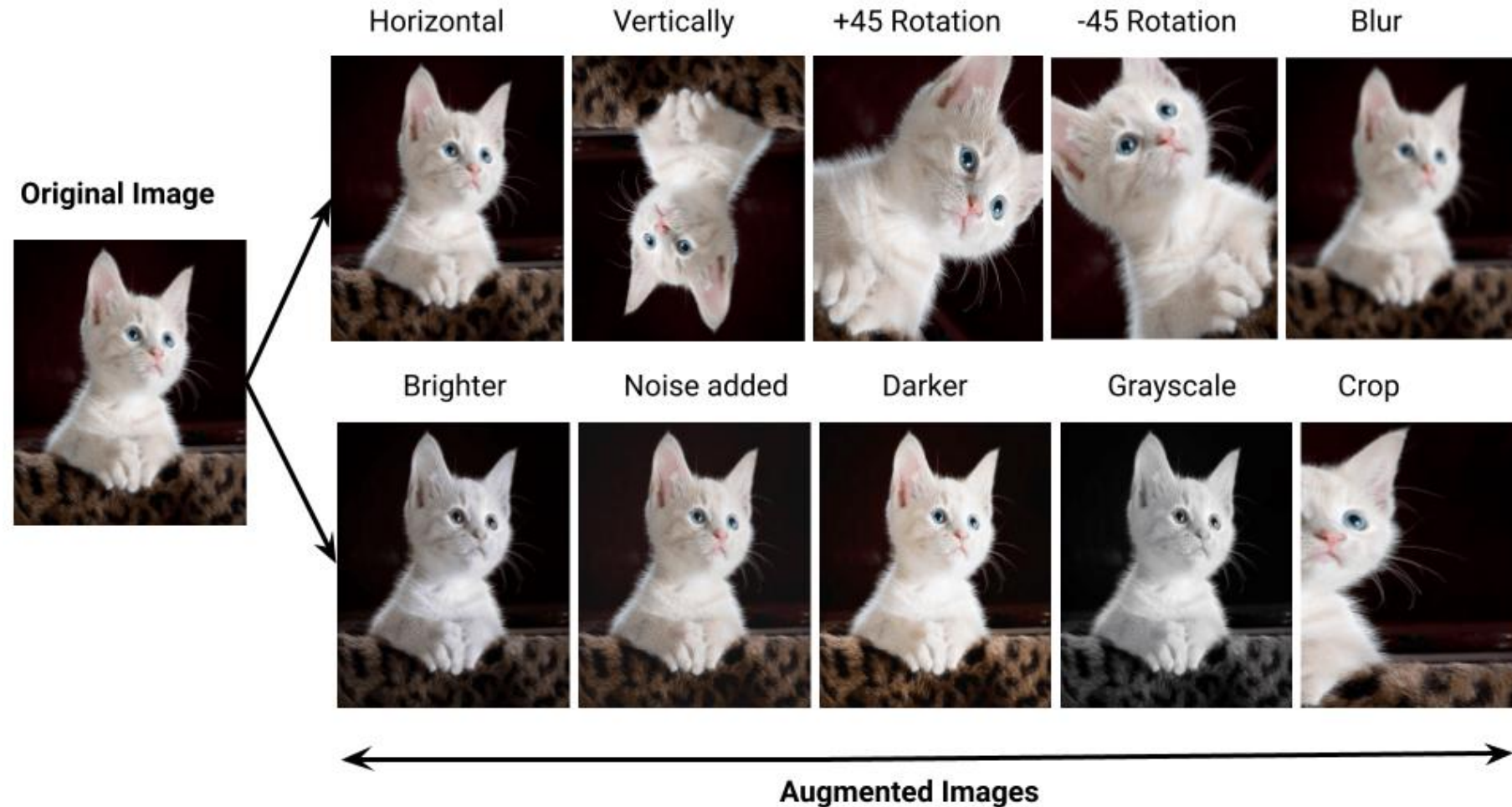


Methods of Regularization

- L^2 – regularization or Weight Decay: $L_{\text{with regularization}} = L_{\text{original}} + \lambda \|w\|_2$
- Early stopping:

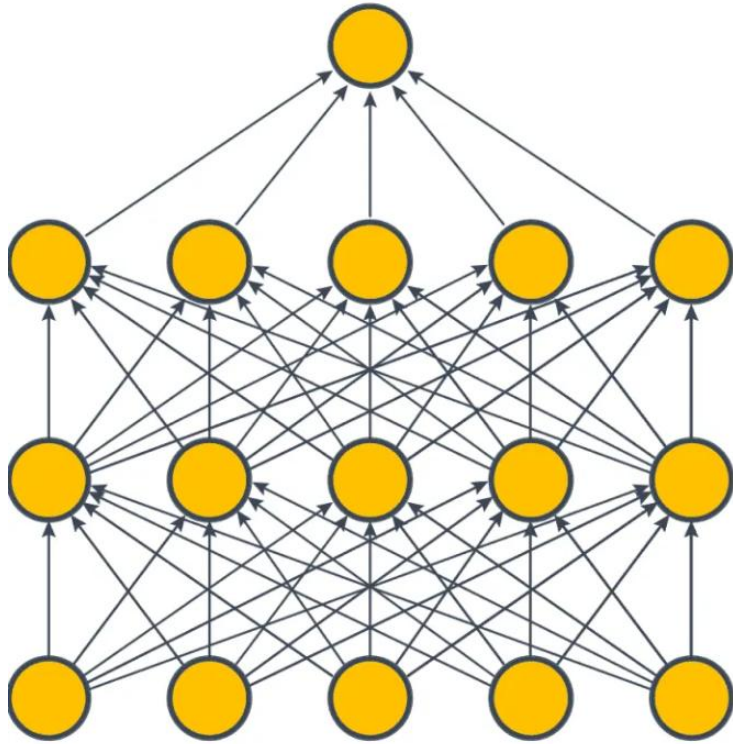


Methods of Regularization: Data Augmentation

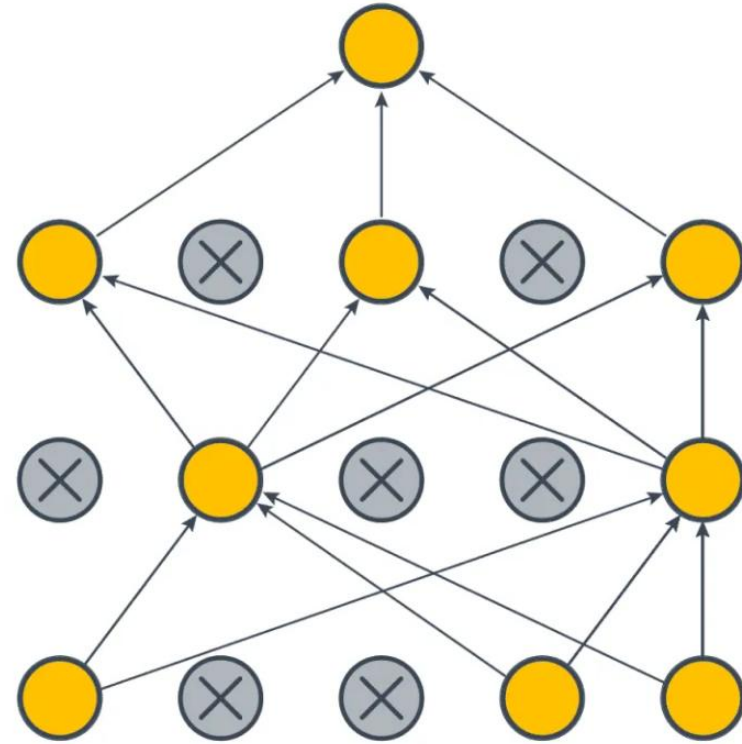


Methods of Regularization: Dropout

Standard Neural Net



After applying dropout



p – probability of an element to be zeroed

During training: $x^{k+1} = \frac{1}{1-p} x^k \odot mask$, $mask \sim Bernoulli(1 - p)$